

Mongo db cheatsheet



BASE DATASET

```
1 db.orders.insertMany([
2   { _id: 1, customer: "A", product: "Pen", amount: 100, city:
3     "Bangalore", tags: ["stationery"] },
4   { _id: 2, customer: "A", product: "Book", amount: 200, city:
5     "Bangalore", tags: ["education"] },
6   { _id: 3, customer: "B", product: "Pen", amount: 300, city:
7     "Delhi", tags: ["stationery"] },
8   { _id: 4, customer: "B", product: "Book", amount: 150, city:
9     "Delhi", tags: ["education"] },
10  { _id: 5, customer: "C", product: "Pen", amount: 50, city:
11    "Mumbai", tags: ["stationery"] },
12  { _id: 6, customer: "C", product: "Book", amount: 400, city:
13    "Mumbai", tags: ["education"] }
14 ])
```



PART 1: FIND + FILTER + PROJECTION

◆ BASIC FIND

```
1 db.orders.find({ customer: "A" })
```

👉 filter = {}

◆ CONDITIONS (VERY IMPORTANT)

```
1 db.orders.find({ amount: { $gt: 150 } })
```

Operator	Meaning
\$gt	>
\$lt	<
\$gte	≥
\$lte	≤
\$ne	not equal

◆ MULTIPLE CONDITIONS

```
1 db.orders.find({
2   city: "Delhi",
3   amount: { $gt: 100 }
4 })
```

👉 AND condition (default)

◆ OR CONDITION

```
1 db.orders.find({
2   $or: [
3     { city: "Delhi" },
4     { amount: { $lt: 100 } }
5   ]
6 })
```

◆ PROJECTION (SELECT FIELDS)

```
1 db.orders.find(
2   { customer: "A" },
3   { customer: 1, amount: 1, _id: 0 }
4 )
```

👉 1 = include

👉 0 = exclude

⚠️ RULE

👉 Cannot mix include & exclude (except `_id`)

🧠 PART 2: SORT / SKIP / LIMIT

◆ SORT

```
1 db.orders.find().sort({ amount: -1 })
```

◆ LIMIT

```
1 db.orders.find().limit(2)
```

◆ SKIP (pagination)

```
1 db.orders.find().skip(2).limit(2)
```

🔥 COMBINED (VERY IMPORTANT)

```
1 db.orders.find({ amount: { $gt: 100 } })
2   .sort({ amount: -1 })
3   .skip(1)
4   .limit(2)
```

👉 Order matters:

1. filter
 2. sort
 3. skip
 4. limit
-

🧠 PART 3: UPDATE (SUPER IMPORTANT)

🔥 STRUCTURE

```
1 db.collection.updateOne(
2   { filter },
3   { update operators }
4 )
```

◆ 1. \$set

```
1 db.orders.updateOne(
2   { _id: 1 },
3   { $set: { amount: 120, city: "Chennai" } }
4 )
```

◆ 2. \$inc

```
1 db.orders.updateMany(  
2   {}  
3   { $inc: { amount: 10 } }  
4 )
```

◆ 3. MULTIPLE OPERATORS

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   {  
4     $set: { city: "Delhi" },  
5     $inc: { amount: 20 }  
6   }  
7 )
```

◆ 4. \$push (array add)

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $push: { tags: "sale" } }  
4 )
```

◆ 5. \$pull (array remove)

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $pull: { tags: "stationery" } }  
4 )
```

◆ 6. \$addToSet (no duplicates)

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $addToSet: { tags: "sale" } }  
4 )
```

◆ 7. \$unset (REMOVE FIELD)

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $unset: { city: "" } }  
4 )
```

👉 field deleted

◆ 8. UPDATE NESTED FIELD

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $set: { "address.city": "Bangalore" } }  
4 )
```

◆ 9. UPSERT (VERY IMPORTANT)

```
1 db.orders.updateOne(  
2   { customer: "Z" },  
3   { $set: { amount: 500 } },  
4   { upsert: true }  
5 )
```

👉 creates document if not found

⚠ COMMON UPDATE MISTAKES

- ✗ forgetting `$set` → replaces whole doc
 - ✗ duplicate `$set`
 - ✗ not using `$inc` for numeric updates
-

🧠 PART 4: DELETE

◆ DELETE ONE

```
1 db.orders.deleteOne({ _id: 1 })
```

◆ DELETE MANY

```
1 db.orders.deleteMany({ amount: { $lt: 100 } })
```

⚠ DANGEROUS

```
1 db.orders.deleteMany({})
```

💀 deletes everything

🧠 PART 5: AGGREGATION (CORE)

🔥 BASIC STRUCTURE

```
1 db.orders.aggregate([
2   { $match: { ... } },
3   { $group: { ... } },
4   { $sort: { ... } },
5   { $limit: N }
6 ])
```

◆ 1. \$match (filter)

```
1 { $match: { amount: { $gt: 100 } } }
```

◆ 2. \$group

```
1 {
2   $group: {
3     _id: "$customer"
4     total: { $sum: "$amount" },
5     avg: { $avg: "$amount" },
6     min: { $min: "$amount" },
7     max: { $max: "$amount" },
8     count: { $sum: 1 }
9   }
10 }
```

◆ 3. \$sort

```
1 { $sort: { total: -1 } }
```

◆ 4. \$limit

```
1 { $limit: 2 }
```

🔥 FULL PIPELINE EXAMPLE

👉 “Top 2 customers with amount > 100”

```
1 db.orders.aggregate([
2   { $match: { amount: { $gt: 100 } } },
3   {
4     $group: {
5       _id: "$customer"
6       total: { $sum: "$amount" }
7     }
8   },
9   { $sort: { total: -1 } },
10  { $limit: 2 }
11 ])
```

🧠 IMPORTANT INSIGHT

👉 \$match before \$group = faster

👉 \$sort after \$group = logical

⚡ PROJECTION IN AGGREGATION

```
1 {
2   $project: {
3     customer: "$_id",
4     total: 1,
5     _id: 0
6   }
7 }
```

🧠 FINAL MASTER FLOW

FIND FLOW

```
1 find → project → sort → skip → limit
```

AGG FLOW

```
1 match → group → sort → limit → project
```

HIGH-YIELD TIPS (VERY IMPORTANT)

1. Always filter early

```
1 $match first
```

2. Use `$sum: 1` for count

3. Use `$addToSet` for unique values

4. Never forget `$set`

5. Combine operators smartly

```
1 {  
2   $set: { ... },  
3   $inc: { ... },  
4   $push: { ... }  
5 }
```

6. Use sort + limit for “top K”

